

SE 811: Software Maintenance

Toukir Ahammed

Corrective Change

- Corrective change refers software modifications initiated by software defects.
- A defect can result from design errors, logic errors, and coding errors
 - **Design errors:** software design does not fully align with the requirement specification. The faulty design leads to a software system that either incompletely or incorrectly implements the requested computational functionality.
 - **Logic errors:** a program behaves abnormally by terminating unexpectedly or producing wrong outputs. The abnormal behaviors are mainly due to flaws in software functionality implementations.
 - **Coding errors:** although a program can function well, it takes excessively high runtime or memory overhead before responding to user requests. Such failures may be caused by loose coding or the absence of reasonable checks on computations performed.

Applying Changes

- Corrective changes such as bug fixes are frequently applied by developers to eliminate defects in software. There are mainly two lines of research conducted:
 1. empirical studies to characterize bugs and corresponding fixes and
 2. automatic approaches to detect and fix such bugs.

Categories of Bugs

Dimension	Category	Description
Root Cause	Memory	Bugs caused by improper handling of memory objects.
	Concurrency	Bugs that happen only in multi-threading (or multi-processes) environment, including data race, deadlock, and synchronization.
	Semantic	Inconsistent with the original design requirements or the programmers' intention. We consider all bugs as Semantic bugs unless they are already classified as Memory bugs or Concurrency bugs.
Impact	Hang	Program keeps running but does not respond.
	Crash	Program halts abnormally.
	Data Corruption	Mistakenly change user data.
	Performance Degradation	Functions correctly but runs/responds slowly.
	Incorrect Functionality	Not behave as expected.
	Unknown	The impact cannot be identified from the bug report.
Software Component	Core	Bugs related to core functionality implementations.
	GUI	Bugs related to graphical user interfaces.
	Network	Bugs related to network environment and network communication.
	I/O	Bugs related to I/O handling.

Source: Li, Zhenmin, et al. "Have things changed now? An empirical study of bug characteristics in modern open source software." Proceedings of the 1st workshop on Architectural and system support for improving software dependability. 2006.

Categories of Bugs

Memory Bug	Memory Leak	Failures to release unused memory.
	Uninitialized Memory Read	Read memory data before it is initialized.
	Dangling Pointer	Pointers still keep freed memory addresses.
	NULL Pointer Dereference	Dereference of a null pointer.
	Overflow	Illegal access beyond the buffer boundary.
	Double Free	One memory location is freed twice.
Semantic Bug	Missing Features	A feature is supposed to be but is not implemented.
	Missing Cases	A case in a functionality is not implemented.
	Corner Cases	Some boundary cases are considered incorrectly or ignored.
	Wrong Control Flow	The control flow is incorrectly implemented.
	Exception Handling	Do not have proper exception handling.
	Processing	Processing such as evaluation of expressions and equations is incorrect.
	Typo	Typographical mistakes.
	Other Wrong Functionality Implementation	Any other semantic bug that does not meet the design requirement.

Source: Li, Zhenmin, et al. "Have things changed now? An empirical study of bug characteristics in modern open source software." Proceedings of the 1st workshop on Architectural and system support for improving software dependability. 2006.

Rule-Based Bug Detection and Fixing Approaches

- Rule-based bug detection approaches detect and fix bugs based on the assumption that bugs are deviant program behaviors that violate implicit programming rules.
- Where are those implicit rules coming from?
 - Such rules can be written by the developers of bug-finding tools or can be refined based on empirical observation in the wild.
 - For example, “release locks after acquiring them”
- If any code snippet violates the specified rule(s), the approach reports the snippet as a software bug

Sample system rule templates and examples

Rule template	Example
“Never/always do X”	“Do not use floating point in the kernel”
“Do X rather than Y”	“Use memory mapped I/O rather than copying”
“Always do X before/after Y”	“Check user pointers before using them in the kernel”

Automated Repair

- Automatic program repair generates candidate patches and checks correctness using compilation, testing, and/or specification.
- search-based repair
- predefined repair templates
- For example, GenProg generates candidate patches by replicating, mutating, or deleting code randomly from the existing program
 - uses genetic programming (GP) to search for a program variant that retains required functionality but is not vulnerable to the defect in question
- As another example, PAR encodes ten common bug fix patterns from Eclipse JDT's version history to improve GenProg.